PRAKTIKUM
MSIM4401

Aktivitas 8: Praktikum Struktur Direktori dan elemen Ionic menggunakan Vue

Kani

Praktikum membangun Struktur Direktori Dan Elemen Ionic Menggunakan Vue

Aktivitas

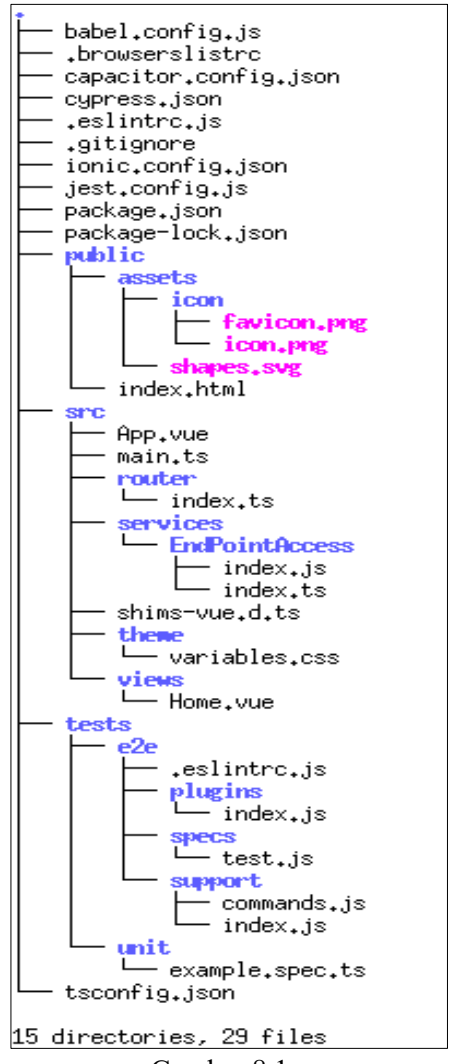
8

A. LEBIH RINCI DENGAN STRUKTUR DIREKTORI DAN FILE

Secara umum, struktur direktori dan berbagai file pada proyek Ionic berbasis Vue terbagi menjadi 2:

1. Struktur direktori dan berbagai file yang bukan *source code*, berada pada direktori akar atau *root* dari proyek.
2. Struktur direktori dan berbagai file kode sumber, berada pada direktori *src*.

Bagung struktur direktori dan file lengkap (dengan mengecualikan *node_modules* yang berisi berbagai *library* yang diperlukan) pada suatu proyek Ionic berbasis Vue adalah sebagai berikut:



Gambar 8.1
Gambar Struktur Direktori
(Asumsi dengan Proyek baru)

Penjelasan Dasar, jika mahasiswa sudah mengerti bisa di abaikan:

1) Berbagai file **konfigurasi**:
   a. *babel.config.js*: konfigurasi Babel. Babel merupakan transpilator JavaScript, artinya *software* yang digunakan untuk mengubah versi terkini dari JavaScript ke versi lama untuk keperluan kompatibiltas. Informasi lebih lanjut: https://babeljs.io/.
   b. *.browserlistrc*: konfigurasi untuk berbagai browser target serta versi Node.js untuk berbagai *frontend tool.* Informasi lebih lanjut: https://github.com/browserslist/browserslist.

c. *capacitor.config.json*: konfigurasi untuk *Capacitor*, *framework* untuk akses API *device* secara *native*. Bagian ini akan kita bahas lebih mendalam di modul-modul mendatang.
d. *cypress.json*: konfigurasi untuk *Cypress*. *Cypress* digunakan sebagai *software* untuk *testing*. Info lebih lanjut: https://www.cypress.io/.
e. *eslintrc.js*: konfigurasi untuk linter - ESLint.
f. *.gitignore*: konfigurasi untuk Git di GitHub, fungsinya untuk mengecualikan berbagai file serta direktori yang ada pada file ini dari proses *push* ke GitHub.
g. *ionic.config.json*: konfigurasi Ionic.
h. *jest.config.js*: konfigurasi untuk Jest. Jest adalah *testing framework*. Info lebih lanjut: https://jestjs.io/.
i. *package.json* dan *package.json.lock*: konfigurasi untuk proyek berbasis Node.js. Paket akan diletakkan di *node_modules*, sementara perubahan-perubahan yang terjadi pada *node_modules* maupun *package.json* akan disimpan pada *package.json.lock*.
j. *tsconfig.json*: konfigurasi untuk kompilator TypeScript.

2) Direktori ***node_modules***: berbagai pustaka yang diperlukan oleh proyek diletakkan pada direktori ini.
3) Direktori ***public***: berisi berbagai direktori dan file yang akan digunakan oleh Webpack untuk membentuk hasil akhir.
4) Direktori ***tests***: berisi berbagai kode sumber serta aset yang digunakan untuk membuat *testing*.
5) Direktori ***src***: berisi berbagai kode sumber serta aset yang digunakan oleh hasil akhir *software* yang dibangun.

Source code proyek diletakkan di direktori *src*. Di dalam *src* terdapat berbagai file dan direktori. Pada umumnya, pemrogram Vue menggunakan direktori dan berbagai *file* berikut:

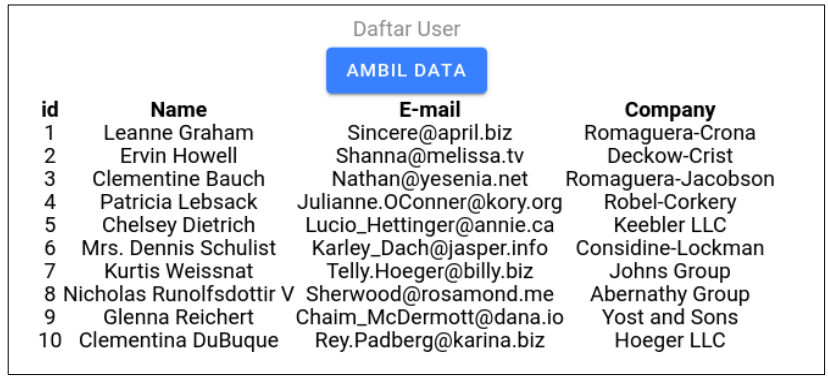
1. ***App.vue***: komponen halaman utama
2. Direktori ***components***: berisi berbagai definisi komponen dalam suatu *single file components*. Setiap satu komponen menempati satu file .vue.
3. ***main.ts***: berisi kode sumber TypeScript utama dari aplikasi.
4. ***shims-vue.d.ts***: berisi berbagai definisi TypeScript untuk proyek Vue.
5. Direktori ***router***: berisi berbagai definisi rute.
6. **theme**: berisi berbagai definisi CSS yang digunakan oleh aplikasi
7. ***views***: berisi berbagai tampilan yang digunakan di proyek.

B. PRAKTIK: TARGET TAMPILAN DAN *LOGIC*

Buatlah Proyek baru dan bangunlah seperti pada Gambar 8.2.

1) **Tujuan Aplikasi**

Aplikasi ini akan menampilkan semua data *user* (beberapa *field* tertentu saja) yang berasal dari https://jsonplaceholder.typicode.com/users jika tombol di-klik.



Gambar 8.2
Gambar Struktur Direktori

Untuk membuat aplikasi ini, sebenarnya bisa saja di halaman tersebut langsung diberikan *source code* untuk mengambil *JSON* dari *URL* dan kemudian ditampilkan di *template*. Meskipun demikian, hal ini kurang baik dilakukan jika di kemudian hari kita akan membangun halaman lain yang memerlukan akses data dari *RESTful API* seperti itu.

Dengan demikian, kita perlu membuat suatu *class* yang diletakkan pada direktori *services* dan saat akan mengambil data dari suatu *endpoint*, kita bisa membuat *instance* dari *class* tersebut. Demikian juga jika kita ingin menambahkan fungsionalitas (tidak hanya *GET*, tetapi bisa juga *POST*), maka kita bisa melakukannya pada *class* tersebut. Dengan demikian, kode sumber kita menjadi *reusable* dan *scalable*.

Definisi *class* di *src/services/EndPointAccess/index.ts* adalah sebagai berikut:

```
import axios from 'axios';                          // <1>

export default class EndPointAccess {               // <2>
 theUrl: string;
 constructor(url: string) {                         // <3>
   this.theUrl = url;
```

```
}

async getRes() {                                    // <4>
  const response = await axios.get(this.theUrl);
  return response;
}
}
```

Listing 8.1
Kode Program file index.ts

Penjelasan bagian //<n>:

1. Menggunakan *axios* (*library* yang digunakan untuk *request* HTTP yang bersifat *open source* dan banyak digunakan saat ini karena memiliki banyak kelebihan dan kemudahan dalam mengaplikasikan.
2. Definisi *class* yang memanfaatkan *axios*.
3. Konstruktor untuk menetapkan URL dari *RESTful API endpoint* yang akan diakses.
4. *Method* untuk *request* GET ke *endpoint*.

Untuk melengkapi proyek, perlu di-*install axios* dan *@types/axios* menggunakan npm dengan perintah:

```
npm install --save-dev axios @types/axios
```

Setelah itu, pada halaman (*src/Home.vue*)yang akan mengakses *RESTful API endpoint*, *class* tersebut bisa digunakan:

```
<template>
 <ion-page>
   <ion-header :translucent="true">
     <ion-toolbar>
       <ion-title>Blank</ion-title>
     </ion-toolbar>
   </ion-header>

   <ion-content :fullscreen="true">
     <div id="container">
       <p>Daftar User</p>
       <p><ion-button @click="ambilData">Ambil data</ion-
button></p>
       <table class="center">
         <tr>
           <th>id</th>
           <th>Name</th>
```

```
          <th>E-mail</th>
          <th>Company</th>
        </tr>
        <tr v-for="user in dataUsers" :key="user.id">
   // <1>
          <td>{{ user.id }}</td>
          <td>{{ user.name }}</td>
          <td>{{ user.email }}</td>
          <td>{{ user.company.name }}</td>
        </tr>
      </table>
    </div>
   </ion-content>
 </ion-page>
</template>

<script lang="ts">
import { IonContent, IonHeader, IonPage, IonTitle, IonToolbar,
IonButton } from '@ionic/vue';
import { defineComponent } from 'vue';
import EndPointAccess from '@/services/EndPointAccess';
   // <2>

let resData: any;                                          //
<3>

export default defineComponent({
 name: 'Home',
 data() {
   return {
     dataUsers: null                                       //
<4>
   }
 },
 methods: {
   ambilData() {
   // <5>
   resData = new
EndPointAccess('https://jsonplaceholder.typicode.com/users');
     resData.getRes()
       .then((response: any)  => (this.dataUsers =
response.data))
   }
 },
 components: {
   IonContent,
   IonHeader,
   IonPage,
   IonTitle,
   IonToolbar,
   IonButton
```

```
 }
});
</script>

<style scoped>
#container {
 text-align: center;
  position: absolute;
 left: 0;
 right: 0;
 top: 50%;
 transform: translateY(-50%);
}

#container strong {
 font-size: 20px;
 line-height: 26px;
}

#container p {
 font-size: 16px;
 line-height: 22px;
  color: #8c8c8c;
  margin: 0;
}

#container a {
 text-decoration: none;
}

.center {
   // <6>
 margin-left: auto;
 margin-right: auto;
}

</style>
```

Listing 8.2
Kode Program file Home.vue

Penjelasan bagian //<n> sebagai berikut:

1. Menampilkan hasil dari *GET* ke *RESTful API endpoint*.
2. Meng-*import class* yang sudah dibuat supaya bisa digunakan di halaman ini.

3. Menyiapkan variabel yang akan digunakan untuk menerima *response* dari *GET* ke *endpoint*.
4. Menyiapkan variabel untuk data *users*.
5. *Method* yang akan dikerjakan jika tombol di-klik yaitu *ambilData*. Bagian ini akan memanfaatkan *service* yang sudah didefinisikan di atas.
6. Definisi *center* untuk menengahkan tabel.

C. *ROUTER* DAN NAVIGASI

Router digunakan untuk mengarahkan akses berdasarkan pada pola URL tertentu. *Router* disatukan di dalam direktori *src/router.*

```
import { createRouter, createWebHistory } from '@ionic/vue-router';
import { RouteRecordRaw } from 'vue-router';
import Home from '../views/Home.vue'                    // <1>

const routes: Array<RouteRecordRaw> = [
 {
   path: '/',
   redirect: '/home'
 },
 {
   path: '/home',
   name: 'Home',
   component: Home                                      // <2>
 }
]

const router = createRouter({
 history: createWebHistory(process.env.BASE_URL),
 routes
})

export default router
```

Listing 8.3
index.ts (src/router)

Penjelasan bagian //<n> sebagai berikut:

1. Halaman yang akan menangani suatu *route* tertentu di-*import* di file index.ts.
2. Pada bagian *component* digunakan untuk memberitahu aplikasi bahwa halaman atau komponen yang akan menangani route */home* adalah *view Home*. Jadi jika pemakai mengakses /home, maka akan ditampilkan halaman seperti yang telah didefinisikan oleh Home (*Home.vue*).